3. docker images:

To list all the docker images pulled on the system with image details such as TAG/IMAGE ID/SIZE etc.

```
[ base  sateeshsahu@Sateeshs-MacBook-Air  ~  docker images

IMAGE                          ID             DISK USAGE   CONTENT SIZE   EXTRA
ddd-book:ch8.1                 2ddd1d0272a3        216MB         51.9MB    U
httpd:latest                   331548c5249b        207MB         47.8MB
mysql:latest                   24e450bbd24f       1.28GB          278MB
satish9336/php-1234:latest     25ca1a259397        714MB          168MB    U
 base  sateeshsahu@Sateeshs-MacBook-Air  ~ 
```

4. docker run:

*•The **'docker run'** command is used to create and start a new container from a **Docker** image. The basic syntax of the **'docker run'** command is as follows:*
•docker run [OPTIONS] IMAGE [COMMAND] [ARG…]
•where,
***•OPTIONS:** optional flags that configure the behaviour of the container, such as setting environment variables, mounting volumes, etc.*
***•IMAGE:** the name of the Docker image to be run.*
***•COMMAND:** the command to be executed when the container is started. If not provided, the default command specified in the Docker image will be used.*
***•ARG -d means:** optional arguments passed to the command.*
•Some commonly used options for executing the docker run command inside a container:
•-it: Runs the container in interactive mode, allowing you to interact with the container through the command line.
•-d: Runs the container in the background.
•-- name: Specifies a name for the container.
• -- rm: Automatically removes the container when it exits.
•-p: Maps a host port to a container port.
•-e / --env: Sets an environment variable inside the container.
•-v: Mounts a host volume inside the container

```
[ base  sateeshsahu@Sateeshs-MacBook-Air  ~  docker run httpd echo "hello docker"
hello docker
```

This command runs a new container from the 'httpd ' image and executes the 'echo' command with the argument "Hello, World!".

—>Running a Container with a Specific Name:
•$ docker run --name my-container httpd echo "Hello, World!"
•--name : this option is to give the name of container
•my-container: it is the name of container
•Echo: it is the command
•"hello,world!" – it is argument used
• This command runs a new container with the specified name "my-container" from the httpd image and executes the echo command with the argument "Hello, World!".

Setting Environment Variables:

- *Environment variables are key–value pairs used by applications to:*
- *Read configuration*
- *Store secrets (usernames, passwords)*
- *Control application behavior without changing the image*
- *Environment variables allow us to configure containers dynamically.*

Need of environment variable:

Without -e:
- *Configuration must be hard-coded in the image*
- *Need to rebuild image for every change*

With -e:
- *Same image → different behavior*

- *Secure & flexible configuration*
- *Widely used in microservices & cloud deployments*
- ***docker run -e MY_VAR=value httpd env***
- *This command sets an environment variable 'MY_VAR' to the value "value" and runs a new container from the 'httpd' image.*
- *The 'env' command is executed to display the list of environment variables in the container.*
- *2nd Example: Run a container from Image and display the variable name inside it.*
- *docker run -it -e MY_NAME=Harpreet ubuntu bash*
- *It will open the container and you can check the env variable inside it as:*
- *Echo $ MY_NAME*
- *Kindly note :*
- *MY_NAME is created inside the container*
- *Host system is not affected*

Multiple Environment Variables

- *docker run -e APP_ENV=production -e APP_VERSION=1.0 nginx*
- *Same for MySQL Container*
- *docker run -d *

*-e MYSQL_ROOT_PASSWORD=root123 *
*-e MYSQL_DATABASE=college *
*-e MYSQL_USER=admin *
*-e MYSQL_PASSWORD=admin123 *
mysql:8

- *Without these variables, MySQL container will fail to start*

Passing Environment Variable from Host System

Ex#1:

- ***Step 1:*** *Set variable on host*

export APP_PORT=8080 (Ubuntu)
Powershell - $env:APP_PORT=8080
Check host variable - echo $env:APP_PORT

- ***Step 2:***

docker run -e APP_PORT=$env:APP_PORT nginx env
docker run -e APP_PORT=$APP_PORT nginx env